

WEB DEVELOPMENT (CIE EXAM MASTER GUIDE)

Complete Revision Material covering 100% Syllabus & Lecture Slides — Units 1 & 2

Course: Web Development

Semester 3

Theory (50) + Practical (25) = 75 Marks

Target: CIE Exam

Course Marking & Evaluation Breakdown

Theory Component (Total: 50 Marks)

- CCE (Continuous Evaluation): 15 Marks
- MOOC Course Completion: 15 Marks
- Table Exam: 20 Marks

Practical Component (Total: 25 Marks)

- Internal Practical Exam: 10 Marks
- Practical Performance: 10 Marks
- File / Lab Journal: 5 Marks

UNIT 1: Web Fundamentals, Protocols & Architecture

1.1 Evolution of the Web (Web 1.0 vs 2.0 vs 3.0)

Feature	 Web 1.0 (~1991–2004)	 Web 2.0 (~2004–Present)	 Web 3.0 (Emerging)
Core Concept	Read-Only Web	Read-Write Web	Read-Write-Execute Web
Content Nature	Static HTML pages, non-interactive	Dynamic, user-generated content	AI-driven, semantic, personalized data
Control Model	Centralized (Site owners)	Centralized (Big platforms)	Decentralized (Users / Blockchain)
Key Technologies	Basic HTML, static graphics	AJAX, JS Frameworks, CMS, Social Media	dApps, Smart Contracts, AI Agents, Semantic Web
Real Examples	Personal homepages, static directory sites	Facebook, YouTube, Wikipedia, Blogs	DeFi platforms, NFT markets, Decentralized Identity

1.2 Core Web Terminologies & Client-Server Architecture

- **Web Client:** Any device/program (laptop, smartphone, tablet) that sends HTTP requests to a server to request resources.
- **Web Server:** A 24/7 online machine running software (Apache, Nginx, IIS) that listens for requests, processes logic, and returns responses.
- **Web Browser:** Client software (Chrome, Firefox, Edge, Safari) that sends requests, receives code, and renders (paints) HTML/CSS/JS into a visual page.
- **Webpage vs Website:** A *Webpage* is a single HTML file (`.about.html`). A *Website* is a connected collection of web pages hosted under a single domain.
- **2-Tier Architecture:** Direct communication between Client and Server.
- **3-Tier Architecture:**
 1. **Presentation Tier:** UI in Browser (HTML/CSS/JS)
 2. **Logic Tier:** Server Code (PHP, Python, Node)
 3. **Data Tier:** Database (MySQL, MongoDB)



The HTTP Request-Response Cycle:

User Types URL/Clicks → Browser sends **HTTP Request** (Method + Headers) → Server receives & processes data → Server sends **HTTP Response** (Status Code + HTML/JSON payload) → Browser parses and renders visual UI.

1.3 HTTP Request Methods: GET vs. POST

Comparison Factor	 GET Method	 POST Method
Primary Purpose	Requests / fetches data from server	Sends data to server to create/update resources
Data Payload Location	Appended directly to URL string (`.?id=10&cat=books`)	Enclosed inside the HTTP Request Body
Data Visibility & Security	Visible in address bar (Insecure for passwords)	Hidden from URL address bar (Secure for input)
Data Length Limit	Restricted by maximum URL length (~2048 chars)	No strict length limit; allows large uploads
Caching & Bookmarks	Can be cached, bookmarked, and stored in browser history	Cannot be bookmarked or cached by default
Idempotency	Idempotent (Repeating request produces identical result)	Non-Idempotent (Repeating request creates duplicate entries)

1.4 Secure Connections (HTTP vs HTTPS, SSL & TLS)

HTTP (HyperText Transfer Protocol)

Transmits data as plain unencrypted text over TCP port 80. Highly vulnerable to packet sniffing and Man-in-the-Middle (MitM) eavesdropping attacks.

HTTPS (HTTP Secure)

Encrypts HTTP traffic using **SSL/TLS** over TCP port 443. Guarantees *Confidentiality* (encryption), *Integrity* (no tampering), and *Authentication* (SSL Certificate verification).

The SSL/TLS Handshake Process:

1. **Client Hello:** Browser connects to server and requests secure session.
2. **Server Hello & Certificate:** Server presents its SSL certificate containing its public key.
3. **Key Exchange:** Browser validates certificate with Certificate Authority (CA) and generates a secret symmetric session key encrypted with server's public key.
4. **Encrypted Session Begins:** Server decrypts session key using its private key. All subsequent communications use symmetric encryption.

1.5 Web Addressing: IP Address, DNS, Domain, & URL

- **IP Address:** A unique numeric network identifier assigned to every connected device.
 - **IPv4:** 32-bit numerical format written in dotted decimal (e.g., `192.168.1.10`, ~4.3 billion unique addresses).
 - **IPv6:** 128-bit hexadecimal format separated by colons (e.g., `2001:0db8:85a3::8a2e:0370:7334`) created to fix IPv4 depletion.
 - **Public vs Private IP:** Public IPs are globally routable across the internet; Private IPs are restricted to local networks (LAN).
 - **Static vs Dynamic IP:** Static is manually assigned and fixed; Dynamic is automatically assigned via DHCP and changes periodically.
- **Domain Name:** A human-friendly alias (e.g., `example.com`) mapping to an IP address.
- **DNS (Domain Name System):** The "phonebook of the Internet" that translates human-readable domain names into numerical IP addresses.
- **URL (Uniform Resource Locator):** Complete path specifying protocol, domain, port, and file path:
`https://www.example.com:443/courses/web-dev/index.html`

1.6 Web Application Types, Architectures & Servers

Application Types

- **Static Website:** Fixed HTML/CSS content stored on disk. Same for all visitors. Fast & low cost.
- **Dynamic Website:** Content built dynamically per request using server scripts (PHP) and databases (MySQL).
- **Single Page Application (SPA):** Loads a single HTML shell; dynamically rewrites content via JS AJAX calls without page reloads (e.g., Gmail).
- **Progressive Web App (PWA):** Web app with offline caching, push notifications, and native app feel.

Server Environment Types

- **Local Server:** Runs locally on developer machine (e.g., XAMPP, WAMP, Node localhost). Used for building & private testing before launch.
- **Global / Production Server:** Hosted in commercial data centers, accessible worldwide 24/7 with public static IP & domain name.

UNIT 2: Building User Interfaces (HTML5, CSS3 & UI/UX)

2.1 UI/UX Design Fundamentals & Psychology

The Four Pillars of Great UI Design

1. **User-Centered Design (UCD):** Designing around real user goals, personas, interviews, and usability testing. "Empathy first, pixels second."
2. **Typography Fundamentals:** Type choices, sizing, line-height, hierarchy, link distinction, and cross-page consistency.
3. **Layout & Composition:** Utilizing grids, tables, and lists to structure visual scanning patterns (F-pattern, Z-pattern).
4. **Color Theory & Psychology:** Strategic color choices that direct user focus and evoke targeted emotions.



Color Psychology Associations

- **Blue:** Trust, Security, Professionalism, Calmness (Banks, Tech)
- **Red:** Urgency, Passion, Danger, Energy, Action (CTAs, Warnings)
- **Green:** Growth, Health, Success, Wealth, Nature (Finance, Environment)
- **Yellow:** Optimism, Caution, Warmth, Friendliness (Highlights)

2.2 Complete HTML5 Document Structure & Syntax

HTML (HyperText Markup Language) provides the structural skeleton of a webpage using tags and elements.

```
<!-- Declaration specifying HTML5 standard -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exam Revision Page</title>
  <!-- Linking External CSS -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Welcome to Web Development</h1>
  <p>This is body text with a <br> line break.</p>
</body>
</html>
```

Section: Holds metadata, character encoding, viewport settings, title bar text, external CSS links, and internal styles. Invisible on screen.

Section: Contains all elements rendered visually on screen for visitor interaction (headings, paragraphs, images, forms, buttons).

2.3 Text Elements, Formatting & HTML Entities

Heading & Paragraph Tags

- `<h1>` to `<h6>`: Document heading hierarchy (

highest,

lowest).

- `<p>`: Paragraph text block.
- `<br>`: Void/self-closing element forcing line break.
- `<hr>`: Horizontal thematic break rule.
- `<strong>` / `<b>`: Important bold text.
- `<em>` / `<i>`: Emphasized italic text.
- `<u>` / `<mark>`: Underlined / Highlighted text.

HTML Entities

Escaped codes used to render reserved syntax characters safely as visible text:

Entity	Symbol	Meaning
<code>&lt;</code>	<code><</code>	Less-than sign
<code>&gt;</code>	<code>></code>	Greater-than sign
<code>&amp;</code>	<code>&</code>	Ampersand
<code>&quot;</code>	<code>"</code>	Double quote
<code>&copy;</code>	<code>©</code>	Copyright symbol
<code>&nbsp;</code>		Non-breaking space

2.4 HTML Lists (3 Types)

1. Unordered List (`

```
<ul>
  <li>HTML5 Structure</li>
  <li>CSS3 Styling</li>
</ul>
```

2. Ordered List (`

```
<ol>
  <li>First Step</li>
  <li>Second Step</li>
</ol>
```

3. Description List (`

Used for term-definition pairs or metadata lists:

```
<dl>
  <dt>HTTP</dt>
  <dd>HyperText Transfer Protocol</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

2.5 HTML Tables Structure & Code Example

Nesting Rule: <table> \$ ightarrow\$ (<thead> / <tbody> / <tfoot>) \$ ightarrow\$ <tr> \$ ightarrow\$ <th> or <td>

```
<table>
  <thead>
    <tr><th>Subject Unit</th><th>Exam Weight</th></tr>
  </thead>
  <tbody>
    <tr><td>Unit 1: Web Fundamentals</td><td>6%</td></tr>
    <tr><td>Unit 2: Building UI</td><td>14%</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total Mid Term Scope</td><td>20%</td></tr>
  </tfoot>
</table>
```

2.6 Forms, Controls & Input Types

```
<form action="/process.php" method="POST">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="user_email" placeholder="name@site.com" required>

  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="user_pass">

  <!-- Radio (Exclusive selection) vs Checkbox (Multiple selection) -->
  <input type="radio" id="student" name="role" value="student">
  <label for="student">Student</label>

  <input type="checkbox" id="agree" name="terms">
  <label for="agree">I Agree to Terms</label>

  <!-- Dropdown Selection -->
  <select name="city">
    <option value="anand">Anand</option>
    <option value="vadodara">Vadodara</option>
  </select>

  <textarea name="comments" rows="3"></textarea>
  <button type="submit">Submit Form</button>
</form>
```

2.7 Media, Graphics & Embedded Elements

Images, iframes & Native Media

- ``: Self-closing picture element. `alt` text is critical for accessibility & broken link fallback.
- `<iframe src="page.html"></iframe>`: Embeds another HTML document inside current page.
- `<audio controls><source src="audio.mp3" type="audio/mpeg"></audio>`
- `<video controls poster="thumb.jpg"><source src="clip.mp4" type="video/mp4"></video>`

Graphics Comparison: Canvas vs SVG

Trait	 HTML5 Canvas	 Vector SVG
Rendering	Raster / Pixel-based	Vector-based XML
Drawing	Drawn via JavaScript	Defined directly in DOM
Scalability	Blurs when enlarged	Infinite scaling (no blur)
Use Cases	Games, charts, pixels	Logos, icons, UI shapes

2.8 CSS Integration Types & Selectors Syntax

3 Ways to Attach CSS to HTML

1. **Inline CSS:** Applied directly via style attribute: `<p style="color:red;">` (Lowest reusability).
2. **Internal CSS:** Placed inside `<style>` block inside document `<head>`.
3. **External CSS:** Linked via `<link rel="stylesheet" href="style.css">` (Best industry practice).

CSS Selector Types Reference

Selector Type	CSS Syntax Example	Target Description
Element Selector	<code>p { color: blue; }</code>	Targets every <code><p></code> element on the page
Class Selector	<code>.card { border: 1px solid #ccc; }</code>	Targets all elements with <code>class="card"</code>
ID Selector	<code>#header { background: #333; }</code>	Targets the single unique element with <code>id="header"</code>
Universal Selector	<code>* { box-sizing: border-box; }</code>	Targets every single element on the entire page
Group Selector	<code>h1, h2, h3 { font-family: sans-serif; }</code>	Applies same styles to multiple selectors
Descendant Selector	<code>nav a { text-decoration: none; }</code>	Targets all <code><a></code> tags nested inside a <code><nav></code>

2.9 Page Formatting, Positioning & The CSS Box Model

The CSS Box Model

Every HTML element is a rectangular box consisting of four nested layers (inside to outside):

1. **Content:** Actual text or image (``width` x `height``)
2. **Padding:** Transparent space *inside* the border around content
3. **Border:** Visible or invisible line surrounding the padding
4. **Margin:** Transparent space *outside* the border separating adjacent elements

CSS Positioning (``position``)

- **static:** Default flow. Top/bottom/left/right properties have no effect.
- **relative:** Positioned relative to its normal flow position without affecting surrounding layout.
- **absolute:** Removed from normal flow; positioned relative to nearest positioned ancestor.
- **fixed:** Fixed relative to browser viewport; stays stuck on screen during scrolling.

LAST-MINUTE CIE EXAM REVISION CHEAT SHEET

- **`` vs ``**: Head contains hidden page metadata and styles; Body contains all user-visible UI elements.
- **GET vs POST:** GET puts data in URL query strings (visible, limited, cacheable); POST hides data in Request Body (secure, unlimited, non-cacheable).
- **HTTP vs HTTPS:** HTTPS adds SSL/TLS encryption over port 443 to scrambled data in transit.
- **Box Model Order:** Content $\rightarrow$ Padding $\rightarrow$ Border $\rightarrow$ Margin (from inside out).
- **Tables Structure:** Remember nesting order: `table -> (thead/tbody/tfoot) -> tr -> th/td`.

- **Practice Writing Code by Hand:** Teachers love asking for full HTML form markup and CSS box model diagrams!